CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY & RESEARCH

Department of Information Technology

Subject Name: Data Engineering

Semester: 5th

Subject Code: ITUE301

Academic year: 2026-27 [ODD]

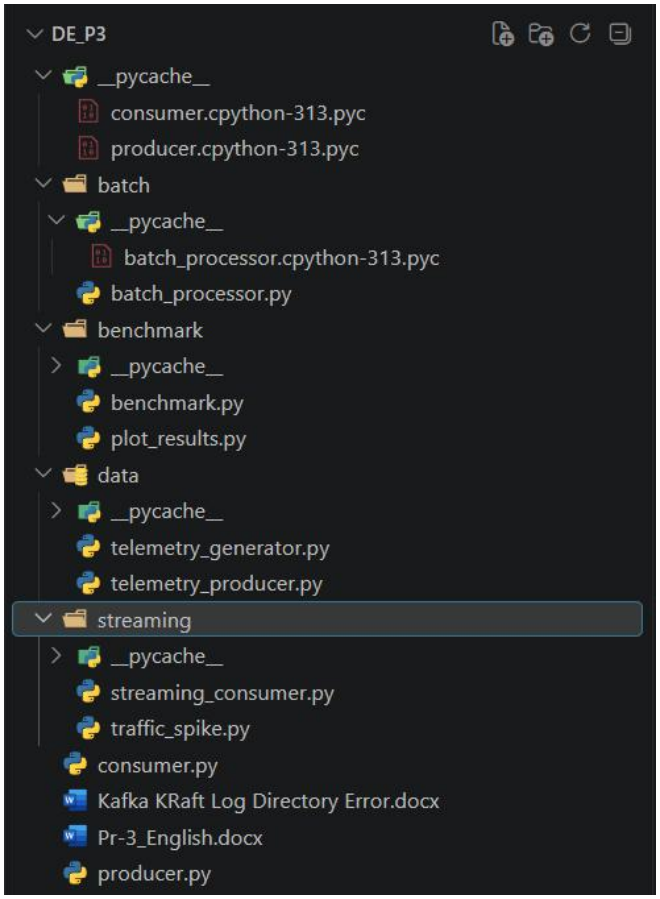
Practical File

PRACTICAL – 3

Problem Definition

Design and implement a real-time data processing pipeline using Apache Kafka. The practical compares streaming and micro-batch processing for telemetry data by measuring processing latency and throughput at different input rates. The system generates telemetry events, sends them through Kafka, processes them using streaming and batch consumers, records benchmark results, and visualizes the performance comparison.

File Structure



Key Questions

1. What is the difference between streaming and micro-batch processing?

Ans:

Streaming processing handles incoming events continuously as they arrive, providing low latency. Micro-batch processing collects events for a fixed interval and processes them together. Therefore, streaming is suitable for real-time applications, while micro-batching can provide better throughput when some delay is acceptable.

2. How does Apache Kafka support real-time data processing?

Ans:

Apache Kafka acts as a distributed event streaming platform. Producers send events to Kafka topics, and consumers read and process those events. Topics can be divided into partitions to support parallel processing and scalability.

3. Which performance parameters are used to compare the two pipelines?

Ans:

The pipelines are compared using:

1. Average processing latency
2. Minimum and maximum latency
3. Processing throughput
4. Number of events generated
5. Number of events successfully processed

Supplementary Problem

Develop and evaluate a Kafka-based telemetry processing system that compares streaming and micro-batch processing at different input rates and identifies the suitable processing approach for low-latency applications.

Solution

The system performs the following operations:

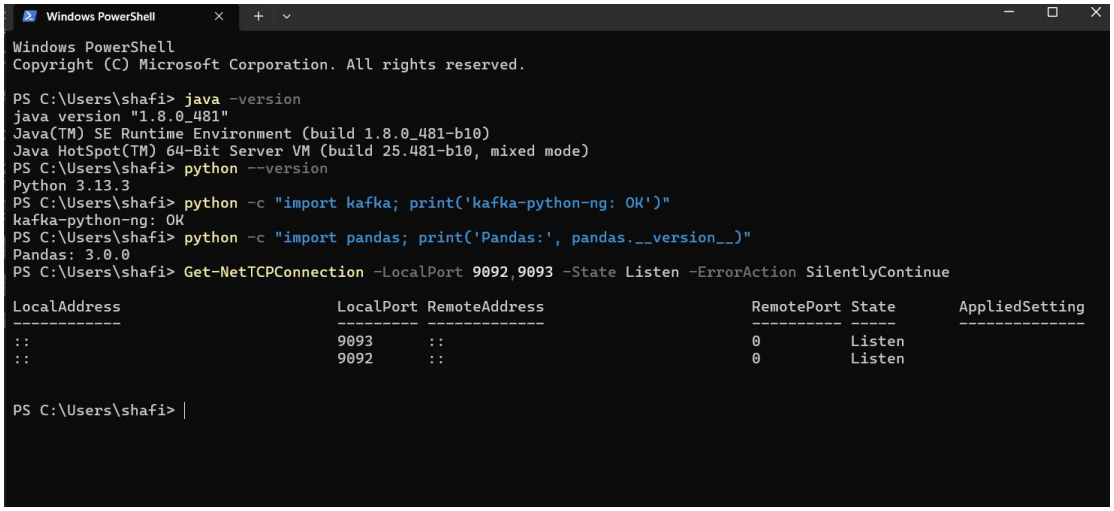
1. Generates telemetry events using a Python producer.
2. Sends the events to the Kafka telemetry topic.
3. Processes events using a streaming consumer.
4. Processes events using a micro-batch processor.
5. Measures latency and throughput.
6. Stores benchmark measurements in CSV files.
7. Generates latency and throughput comparison graphs.
8. Performs a high-rate spike test at approximately 1000 events/sec.
9. Compares the performance of both processing approaches.

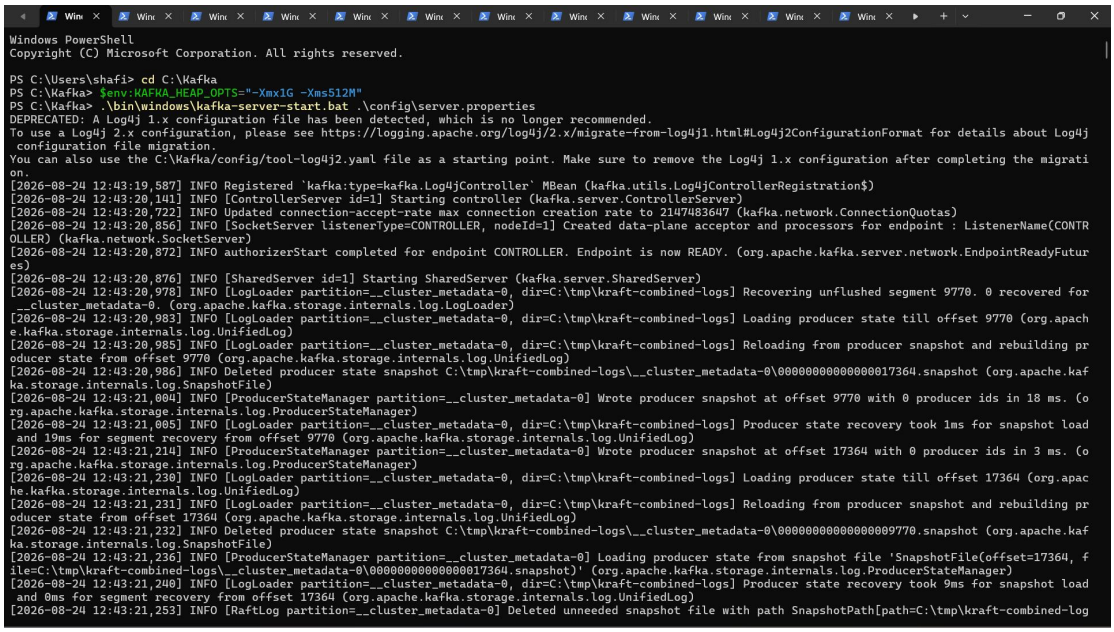
Steps Performed

Step 1: Verify Kafka

Kafka and the required JDK environment were checked, and connectivity to Kafka was verified using port 9092.

Output:
Kafka connection successful.





Step 2: Verify Kafka Topic

The telemetry topic was checked using Kafka topic commands.

The topic contained **4 partitions** with replication factor **1**.

Output:
telemetry topic successfully available.

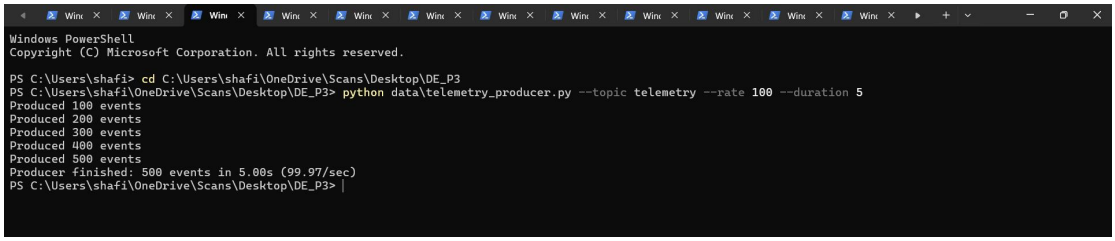


Step 3: Generate Telemetry Data

The producer was executed at 100 events/sec for 5 seconds.

Producer finished: 500 events in 5.00s

Output:
500 telemetry events generated successfully.



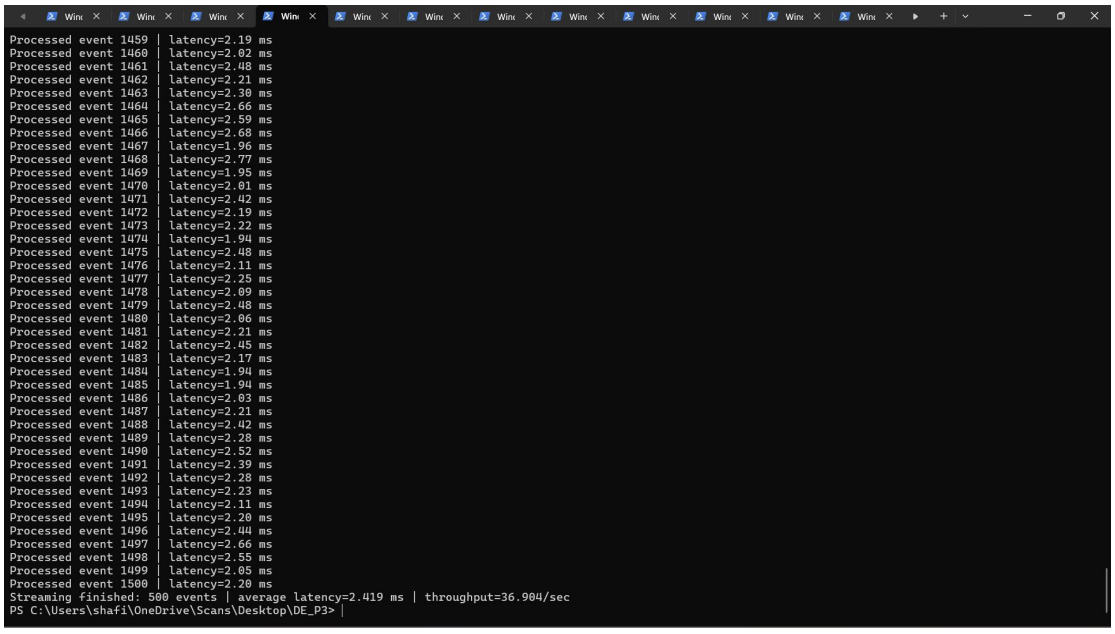
Step 4: Perform Streaming Processing

The streaming consumer was started with a fresh consumer group and processed the generated events.

Streaming finished: 500 events
average latency = 2.419 ms

Another successful run processed 500 events with approximately **2.4 ms average latency**.

Output:
500 events successfully processed using streaming.

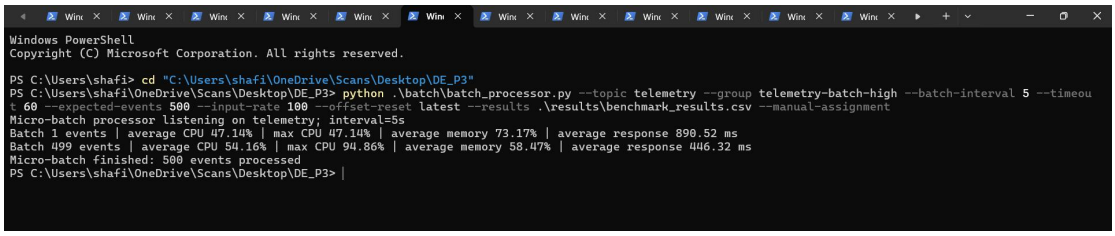


Step 5: Perform Micro-Batch Processing

The micro-batch processor processed the telemetry events using a **5-second batch interval**.

Micro-batch finished: 500 events processed

Output:
500 events successfully processed using micro-batch processing.

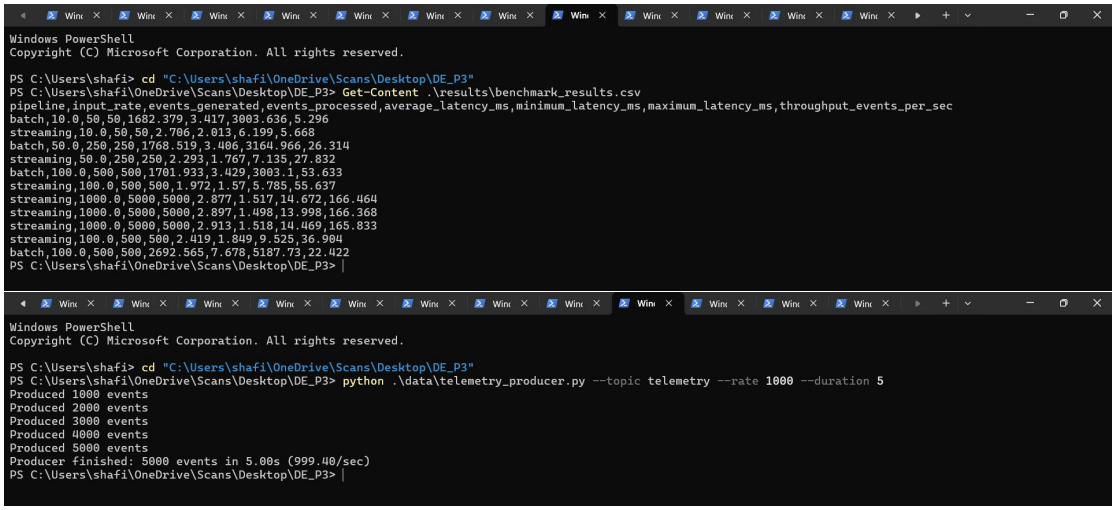


Step 6: Record Benchmark Results

The benchmark results were stored in:

results\benchmark_results.csv

The measurements included input rate, events generated, events processed, average latency, minimum latency, maximum latency and throughput.



Step 7: Generate Performance Graphs

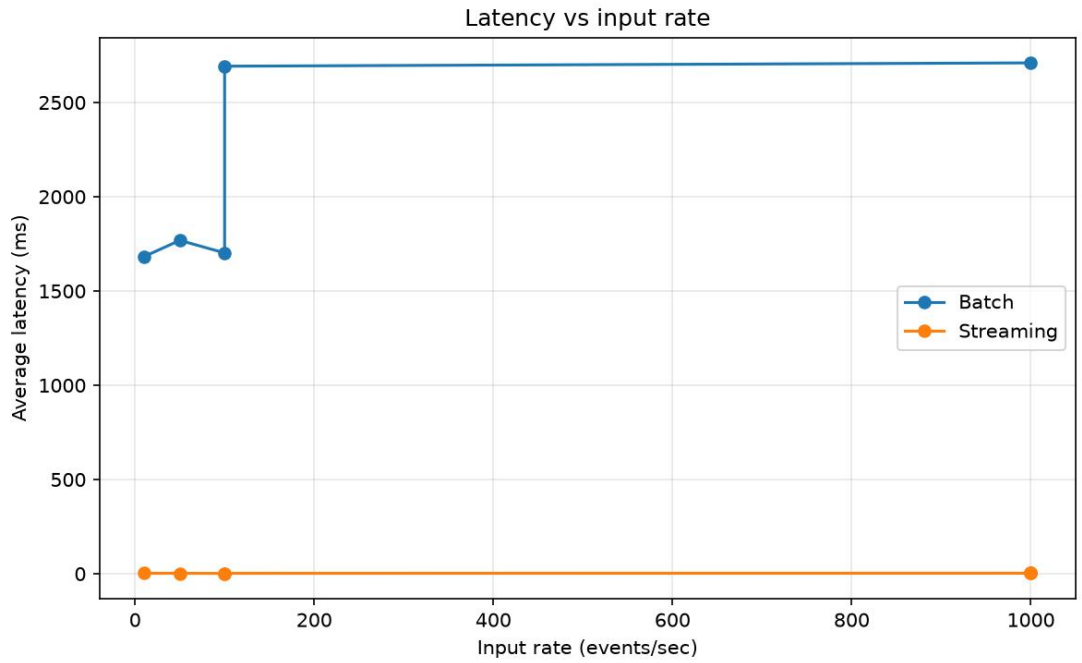
The benchmark plotting script generated:

latency_comparison.png
throughput_comparison.png

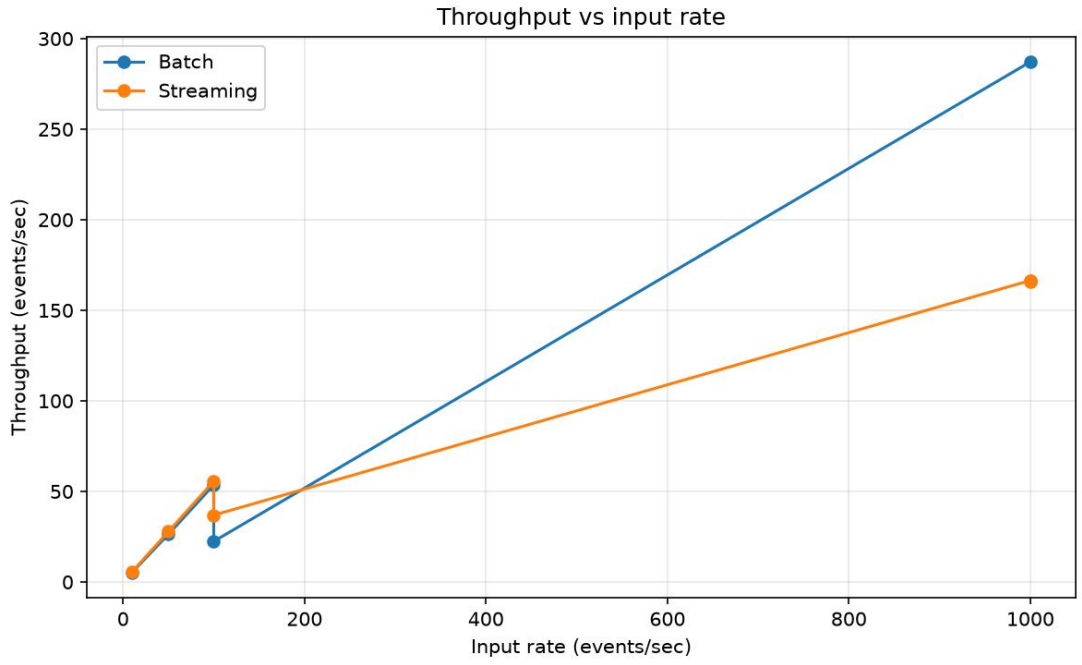
Output:
Performance comparison graphs generated successfully.

Screenshots:

1. Latency vs input rate



2. Throughput vs input rate



Step 8: Perform Spike Test

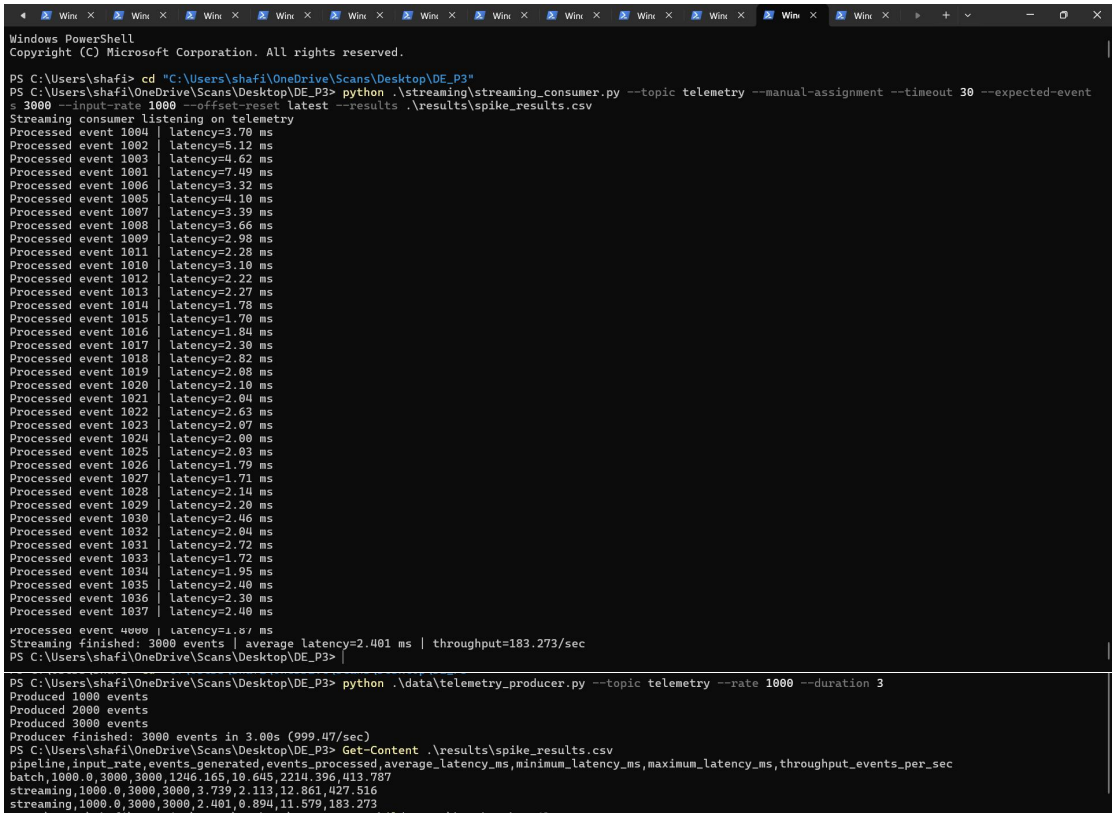
The producer generated 3000 events at approximately 1000 events/sec.

Producer finished: 3000 events in 3.00s

The streaming pipeline successfully processed:

3000 / 3000 events
Average latency = 2.401 ms
Maximum latency = 11.579 ms

Output:
All 3000 events were processed successfully during the spike test.



Conclusion

The practical successfully implemented and compared streaming and micro-batch processing using Apache Kafka. The streaming pipeline achieved significantly lower latency, making it suitable for real-time applications. The micro-batch pipeline introduced higher latency because events were processed periodically in batches. The spike test successfully processed all 3000 generated events, demonstrating reliable event processing at a high input rate. Overall, streaming is preferred for low-latency real-time processing, while micro-batching can be useful when higher throughput is required and some processing delay is acceptable.